server2 structured diagnostics and incident response

This contract describes the repository implementation for #2802. Installation on a production host, production retention and provider availability are not established by local verification. The baseline audit records the pre-change gaps; the implementation audit below records coverage and exclusions. See the flow, editable diagram, PDF and selected verification.

First five minutes of an incident

1. Preserve the UTC incident window, deployed executable SHA-256/release identity, service status and internal metrics. Retain stderr before rotating or replacing the process. Use the installed service's approved log reader; no service name, collector or deployment configuration is assumed here.

2. Find the failed HTTP terminal record by time, `route`, `method` and numeric `status`. Its generated `trace_id` identifies the operation. No client header or user identifier becomes a trace ID. A 501 alone identifies neither a provider failure nor a safe retry.

3. Follow the trace with the local tool below. Read `parent_operation_id` and `operation.linked` records. A joining request has its own trace; `linked_trace_id`/`linked_operation_id` identify the existing owner that performed shared work. Keep both traces in the incident evidence.

4. Read the deepest classified failure, the chosen action and the parent terminal outcome together. HTTP/provider status completion does not confirm a durable commit, and an acknowledged FCM request does not confirm device delivery. Use the decision table before attempting recovery.

5. Check `server2_logging_*_total` and `logging.loss`. Missing start/end/link records, process termination, filtering or drops prevent a complete reconstruction. Preserve uncertainty and inspect authoritative state before a mutating recovery action.

On a copied operational stderr capture, an initial lookup can use:

```
jq -c 'select(.schema==1 and .component=="http" and
  .event=="operation.finished" and .status>=500)' incident.ndjson
python3 server2/tools/diagnose_logs.py --trace TRACE_32_HEX incident.ndjson
# Focus one anonymous submission, retaining ancestor/shared batch context:
python3 server2/tools/diagnose_logs.py --trace TRACE_32_HEX \
  --operation OPERATION_16_HEX incident.ndjson
```

Replace `TRACE_32_HEX` with an observed lowercase 32-digit ID and `OPERATION_16_HEX` with an observed anonymous 16-digit submission ID. The tool reads local files only, follows shared links in both directions and preserves input order and outcomes. It limits input to 128 MiB/100,000 records and rejects oversized or incomplete lines; select a smaller complete UTC window on failure. It reports loss summaries outside the selected trace too. A trace query can include other requests sharing an owner. An operation query follows only that item and its descendants/shared owner, then adds ancestor and shared batch claim/result context; it excludes unrelated sibling sends. A batch result can cover more items than the selected submission, as its `count` states. It never resends, replays, edits state or proves delivery. Search a copied capture rather than modifying the original.

Cause, action and outcome

Evidence	Interpretation	Next diagnostic action
`provider_http`, status 401/403, `auth`	Peer refused authentication/authorization	Check credential presence, expiry and permissions through the approved secret management path. Never print the credential. Parent events distinguish terminal refusal from an existing key rotation.
status 429, `quota`; `reserve` refusal	Provider cap or funding admission refused	Check quota counters and the funding window. A `retry` or `rotate_key` event records an existing decision only; logs do not authorize extra attempts.
status 5xx, `dependency`; `transport`/`timeout` (or a safe connection-refused/reset/address-unavailable class)	Dependency response or transport failure	Follow attempt numbers and the parent's retry/fallback/terminal event. Check the dependency's health without invoking collection routes as probes.
`publish`, `conflict`, `reconcile`	Conditional write conflicted	Follow existing read-back/reconciliation. A transport status receipt is not the committed state.

Evidence	Interpretation	Next diagnostic action
`reconcile`, `reconciled`	Existing reconciliation identified this operation	Confirm the enclosing commit's terminal result. A child read or PUT alone is insufficient.
`void`, `voided`	Existing void compensation was acknowledged	Confirm enclosing commit/reconcile outcome and authoritative journal state. No compensation was added for logging.
`send`, `accepted`	FCM accepted the request	It does not establish device delivery or successful result recording. Follow `record_result` under the same worker trace.
`send`, `unknown`; `do_not_retry`	The send may have reached the external service	Retain the durable claim. Inspect claim/result state through existing authorized procedures; do not replay the notification.
`record_result`, `success`	Result ledger write was recorded	It confirms that ledger operation, not device delivery. `unknown` keeps the retained claim and uncertainty.
`cancelled`, `detach_waiter`	Request waiter left or was dropped	Its owned acquisition may still finish. Follow the linked owner; do not interpret waiter cancellation as owner cancellation.
`publish`, `unknown` on drop	Publication scope ended without acknowledged completion	It conservatively marks possible external work from scope entry, even before a PUT may have started. Reconcile before retrying.
`submission`, `cancelled`	Item processing stopped; an existing batch claim or child send can remain	Read child send outcomes and the batch claim/result. Parent cancellation never proves that no notification reached FCM.
`runtime_teardown`, `unknown`	Completion of detached provider owners was not verified	Shutdown does not attest every provider owner completed. This record alone does not assert that an owner was active or cancelled.
`operation.panic`, `panic`	Panic hook observed an unwind/abort boundary	Use vetted `source`/`line`, build identity and neighboring operation records. Payload/backtrace text is deliberately suppressed.

`completed` means the stage returned; inspect its children/result classifications. `success` applies only to the named stage. `failed` does not imply that every child side effect was undone. `unknown`, `cancelled`, `conflict` and `skipped` are distinct outcomes. No new retry, fallback, journal compensation or FCM resend behavior is introduced. Existing batch ownership and result payloads are preserved.

For an uncertain operator notice mutation, the event includes the CLI-generated `durable_operation_id`. It uses the existing notice-cli nonce/process SHA-256 algorithm and maps the diagnostic trace to the existing transaction. Keep it for authorized durable-state inspection before repeating. Registration IDs, FCM tokens, object keys, arbitrary business identifiers and hashes of personal inputs are not log fields.

Event fields and correlation

Every operational stderr line is schema-1 NDJSON, at most 1,024 bytes including newline. Fields are UTC RFC3339 `timestamp`, caller-declared build revision or `unknown`, severity, fixed `event`, `component`, `stage`, generated trace/operation/parent IDs, safe `route` and `method`, fixed `cause`, `action`, `outcome`, and applicable numeric status/attempt/duration/count. Notification work also carries allowlisted `delivery_kind` (`alarm`, `alert`, `warning`); a failed submission can carry an allowlisted `failure_class` such as weather, message, state guard, credentials or OAuth. No raw error text is copied into that class. Shared links add generated linked IDs. Vetted source identities/line numbers are compiler locations from an allowlist, not user-supplied paths. Generic async wrappers identify their logging boundary; they do not promise a full stack trace.

`build_revision` is populated only by a 40-hex `SERVER2_BUILD_REVISION` compile-time value. It is a caller declaration, not signed provenance; verify the executable hash/release receipt. Timestamps depend on the host clock; compare clock synchronization across hosts. Durations use monotonic time. Operation IDs are unique within a process sequence; use the full trace as the lookup identity across process logs.

HTTP creates a new root without changing response bodies, headers or status. Its terminal/duration records when the router returns a response; it does not confirm body delivery or client receipt. Async tasks, JoinSet futures and blocking work inherit context before handoff. Push writer commands carry enqueue context. Shared acquisitions keep the initiator's owner identity

and link later joiners. Scheduled history/alarm/alert/warning cycles create fresh roots. Each notification item has an anonymous `submission` child spanning preparation/sending; its FCM operation is a child of that item. Alarm/alert claims and result writes remain batch-level, with a count rather than user IDs. Warning claim/result writes belong to the individual submission. Submission acceptance describes the send outcome; follow the applicable result ledger before assessing durability. Mutex-owner metadata is best effort: contention omits a link rather than attributing work to an old owner. Root/terminal records remain useful when an ancestor is filtered or lost.

No event API formats raw errors, URLs, headers, request/response bodies, key material, precise coordinates, registration IDs or notification text. Synthetic privacy checks include URL/header and panic payloads; existing delivery/state tests guard behavior. The stdout readiness line and notice JSON/help output remain machine protocols, not operational stderr. Notice CLI JSON is product output and can contain notice content; keep it separate from the operational log stream.

Output bounds, filtering and retention

- `SERVER2_LOG_LEVEL`: `error`, `warn`, `info` (default), or `debug`. `SERVER2_LOG_RATE`: 1–1,000 events per second per severity; default 100. These are startup settings, not a live configuration endpoint. Invalid values fall back safely and emit a classified configuration notice without the invalid value.

- A fixed 256-record queue feeds one dedicated stderr thread. Producers serialize only bounded typed fields and use try-send/atomics; they never wait for sink IO. Rate admission uses bounded CAS attempts. Context scopes add a small boxed future per boundary to avoid large nested async layouts. No arbitrary output file or network collector is supported.

- Error/warn/info/debug have independent rate buckets. Frequent lookup/restore/reserve/HTTP transport success, expected S3 absence and health/metrics operations are debug. Failures remain warn; panic/startup fatal records are error. Normal client 4xx HTTP terminals are info except 429. A shared queue can still fill and drop errors; separate rate buckets are not a delivery guarantee.

- Internal metrics expose cumulative filtered, rate-limited, queue-full, oversized and sink-error counts. Expected debug filtering is not a failure. Nonzero loss counters mean the capture may be incomplete. The writer tries a `logging.loss` summary at ten-second intervals or a shutdown flush; a blocked/closed sink cannot reliably report its own failure, so metrics are independent.

- Shutdown flushing uses at most 50 ms of unused existing drain allowance. It cannot extend the existing 6-second HTTP-only or 35-second worker process stop bound. Acknowledging a flush is not durable log storage. Runtime teardown can cancel detached provider work after that flush; absence of its terminal event proves nothing.

- Before deployment, assign an approved host collector, bounded storage budget and rotation/retention window, restricted service/operator access, clock synchronization and export procedure. Keep stdout/stderr separate if the host reader supports it. Preserve the selected incident window and executable identity before rotation. No host logging configuration, production retention period or permission change was installed by this task.

Implementation audit

The baseline gap inventory is preserved at its original revision. This table is its implemented counterpart; pure transformations and per-item CPU loops remain intentionally quiet.

Subsystem	Implemented coverage	Boundary/exclusion
HTTP/gateway/listener	Root and terminal status; admission failures; inherited process tasks	No peer IP or raw URL; safe route categories are intentionally broader than endpoint parameters.
Native geocode/world/domestic	Lookup/acquisition/restore; owned async/blocking work and shared links; existing cache/fallback decisions	Pure parsing and per-cell values omitted; completed non-Result stages require child outcomes.
Provider/quota	Actual HTTP status/error class; funded reserve/refusal, attempts, existing retry/rotation	Payload-level rejection may classify as dependency in the parent even when HTTP was 200; no provider body is logged.
S3/catalog/cache	Read/PUT/publication stages; conditional conflict; journal reconciliation/void; cache failure fallback	Transport completion is separate from durable commit; cache skips/misses retain aggregate metrics rather than logging keys.
History/summary/warnings	Cycle/lookup/record stages; inherited owned work, warning refresh shared owners; classified failure counts	No one event per parsed row, warning text or summary value.

Subsystem	Implemented coverage	Boundary/exclusion
Push state writer	Enqueue context; writer, restore, commit, snapshot, reconciliation and existing void	No registration/transaction payload; inability to restore is a failed admission, not a committed update.
Delivery/notice	Fresh worker roots; claim, OAuth, send, result; supervision; translation fallback; safe CLI uncertainty ID	Accepted is not delivered; no resend/replay; CLI stdout JSON/help remains unchanged.
Process/output	Startup/bind/quiesce/drain/teardown; per-component joined booleans; privacy panic hook; drop metrics	Provider completion at runtime teardown remains unverified; stdout readiness retained; host collector/retention excluded.

Local verification

Use a new output directory and explicit retained release executables:

```
python3 server2/tools/logging_smoke.py --binary /path/to/server2 \
  --probe /path/to/logging_smoke --out reports/logging-run \
  --runtime-services /path/to/runtime_services
```

This exercises actual release HTTP and owned loopback 503/403/429 responses, async/blocking propagation, a shared owner, helper cancellation/unknown outcomes and private panic suppression. The drop probes do not prove an external send occurred. With an explicit `--runtime-services` executable, the tool also starts the existing production-lifecycle test entrypoint against owned S3/OAuth/FCM peers, loses one journal write before commit, checks the existing durable void and the next acknowledged update, verifies HTTP-to-writer parent identity, and requires zero notification submissions in that void scenario. A separate two-alarm/one-minute scenario requires exactly two posts to the owned FCM double, distinct anonymous item parents, one existing batch claim/result and no device sends. Omitting that executable records this process proof as not run. The large local domestic Rust target uses the existing owned catalog peer connection-reuse profile to avoid macOS ephemeral-address exhaustion; the other targets use the unchanged default profile, including its explicit HTTP/1.0 compatibility guard. Initial failed runs remain retained separately. Its early-write fault framing is independently regression-checked, without changing product retries or response assertions. Full Rust regressions, actual worker/state/provider failure captures and the all-family suite supply separate business behavior evidence; inspect the selected verification for actual commands, layers, hashes and limitations. This task does not qualify earlier-release whole-service recovery, live shadow, quotas or production. S19 remaining recovery work is separate; #2724 measurements are excluded.

Actual diagnostic capture

Formatted selected fields from the retained NDJSON probe. This includes actual loopback HTTP failures and explicitly labelled helper drop/panic probes; it is not proof of production delivery or deployment.

```
#2802 | actual isolated diagnostics: selected event fields
Formatted excerpt; selected actual fields, not a raw terminal capture
Invocation shape; exact executable hashes are in the report:
python3 server2/tools/logging_smoke.py --binary <retained server2>
  --probe <retained logging_smoke> --out reports/issue-2802/functional-logging4
  --runtime-services <retained runtime_services test executable>

2026-10-17T06:17:47.000561Z
  component=http stage=request
  cause=not_found action=return_response outcome=failed status=404
  trace=af7a33d9dd3c61480000000000000003 op=0000000000000003
2026-10-17T06:17:46.28375Z
  component=provider stage=provider_http
  cause=dependency action=none outcome=failed status=503
  trace=8a6facfc7cf7ca2d0000000000000002 op=0000000000000003
2026-10-17T06:17:46.283844Z
  component=provider stage=provider_http
  cause=auth action=none outcome=failed status=403
  trace=8a6facfc7cf7ca2d0000000000000002 op=0000000000000004
2026-10-17T06:17:46.283943Z
  component=provider stage=provider_http
  cause=quota action=none outcome=failed status=429
  trace=8a6facfc7cf7ca2d0000000000000002 op=0000000000000005
2026-10-17T06:17:46.285398Z
  component=resolver stage=shared_work
  cause=none action=await_existing outcome=pending
  trace=8a6facfc7cf7ca2d000000000000000b op=000000000000000b
  owner=8a6facfc7cf7ca2d0000000000000009 op=000000000000000a
2026-10-17T06:17:46.284015Z
  component=delivery stage=send
  cause=cancellation action=do_not_retry outcome=unknown
  trace=8a6facfc7cf7ca2d0000000000000002 op=0000000000000007
2026-10-17T06:17:46.284025Z
  component=process stage=lifecycle
  cause=panic action=cancel_owner outcome=failed
  trace=8a6facfc7cf7ca2d0000000000000002 op=0000000000000002

PASS actual release HTTP and owned diagnostic probe
Helper drop probes do not establish an external send.
Report SHA-256: 3a96cac1c92a09a175dd92e1b74e88cafcaeb3ba076c7235e0a5bac62e004d15
```

Actual transaction diagnosis

The selected anonymous alarm retains its shared batch claim and result count, while excluding its sibling send. The separate failed-write scenario records existing compensation and the next acknowledged update. All peers are local doubles; FCM acceptance is not device delivery.

```
#2802 | actual owned process transaction evidence
Formatted selected fields; local doubles, no device sends

One selected alarm; other submission sends excluded:
  claim success count=2
    trace=4541106d4ffc266c000000000000001e op=0000000000000025
  send accepted count=-
    trace=4541106d4ffc266c000000000000001e op=000000000000002b
  submission accepted count=-
    trace=4541106d4ffc266c000000000000001e op=0000000000000028
  record_result success count=2
    trace=4541106d4ffc266c000000000000001e op=0000000000000038

Dropped journal write; existing compensation and next ack:
  void cause=ambiguous outcome=voided
    trace=b031d764c09053e20000000000000016 op=000000000000001d
  publish cause=ambiguous outcome=voided
    trace=b031d764c09053e20000000000000016 op=000000000000001a
  writer cause=none outcome=completed
    trace=b031d764c09053e20000000000000016 op=0000000000000017
  publish cause=none outcome=success
    trace=b031d764c09053e20000000000000022 op=0000000000000026
  writer cause=none outcome=completed
    trace=b031d764c09053e20000000000000022 op=0000000000000023

PASS two owned FCM posts; one batch claim, result count=2
PASS void journal=1, acknowledged followup=1; notifications=0
Report SHA-256: 3a96cac1c92a09a175dd92e1b74e88cafcaeb3ba076c7235e0a5bac62e004d15
```